#	Audit Category	Specific Issue Located in Legacy Code	Line(s)	Severity	Impact / Risk
1	Global State	`dataArray`, `currentCount`, and `maxCapacity` are declared as mutable global variables.	8–10	High	Reduces modularity, prevents multiple independent array instances, makes ownership unclear, and creates unsafe shared state.
2	Memory Leak	During dynamic resizing, a new array is allocated and the old `dataArray` is replaced without calling `delete[]`.	18–22	Critical	The original heap allocation becomes inaccessible, producing a memory leak every time the array is resized.

3	Memory Leak	The program terminates without releasing the dynamically allocated `dataArray`.	99–100	High	Heap memory remains allocated until process termination and demonstrates improper resource lifecycle management.
4	Out-of-Bounds Access	`printAll()` uses `i <= currentCount` instead of `i < currentCount`.	58–61	High	Attempts to read one element beyond the valid range, resulting in undefined behavior, garbage output, or a possible crash.
5	Out-of-Bounds Access	`remove_item_at()` checks only whether `idx < 0`; it does not check whether `idx >= currentCount`.	31–34	Critical	Invalid indexes such as 99 can cause memory corruption or invalid memory access.

6	Inefficient Deletion	`remove_item_at()` contains a redundant nested loop and an unnecessary break statement.	37–41	Medium	Makes the deletion algorithm unnecessarily complex and introduces avoidable loop overhead.
7	Inefficient Search	`findItem()` contains an inner loop that executes exactly once.	46–54	Medium	Provides no algorithmic benefit and makes a simple linear search unnecessarily complicated.
8	Naming Standards	Function names use inconsistent naming styles: `add_item()`, `findItem()`, and `printAll()`.	Multiple	Low	Reduces consistency and readability and does not follow a unified naming convention.
9	Encapsulation	Data storage and array-management state are exposed through global variables rather than an encapsulated class or structure.	8–10	High	Any function can directly modify the array state, making the subsystem difficult to maintain and test.

10	Resource Ownership	There is no clearly defined owner responsible for the dynamically allocated array.	Multiple	Critical	Makes memory lifecycle management error-prone and contributes directly to the resize and termination memory leaks.
11	Input Validation	Array operations do not consistently validate indexes before accessing memory.	31–41, 58–61	Critical	Invalid input can cause undefined behavior and memory corruption.
12	Empty-State Handling	Functions do not properly define behavior for operations performed when the array is empty.	Multiple	High	Empty-state operations may access invalid memory or produce unexpected behavior.
13	Hardcoded Matrix Dimensions	`processMatrix()` uses fixed 3 × 3 dimensions and hardcoded loop limits.	66–68, 73–83	Medium	Limits reuse and makes the matrix routine unsuitable for different dimensions without modifying the source code.

14	Poor Modularity	Array management, searching, deletion, printing, matrix processing, and execution logic are implemented without a dedicated data-structure abstraction.	Multiple	Medium	Makes the code harder to extend, test, maintain, and reuse.
15	Error Handling	Invalid deletion only partially validates input and does not protect against upper-bound violations.	31–34	Critical	A negative index is rejected, but a large positive index can still reach unsafe memory operations.
16	Documentation / Maintainability	The legacy implementation does not clearly document ownership, capacity management, or algorithmic assumptions.	Multiple	Low	Makes maintenance and future debugging more difficult for developers.